Unit: 2

Data Warehouse Process and Technology

Warehousing Strategy

A warehouse strategy is a plan that helps you manage your warehouse to maximize efficiency and minimize costs. It can help you:

Improve operations

A warehouse strategy can help you improve your warehouse's efficiency, inventory accuracy, and security.

Identify areas for improvement

A warehouse strategy can help you identify areas to improve, such as bottlenecks, storage density, and workflows.

Reduce costs

A warehouse strategy can help you reduce warehousing and shortage costs.

Increase customer satisfaction

A warehouse strategy can help you reduce lead times and ensure timely deliveries.

Here are some strategies you can use to improve your warehouse:

Implement inventory management systems: These systems can automate tasks like tracking, ordering, and forecasting.

Streamline order fulfillment: You can use order fulfillment software to improve order accuracy and speed.

Invest in technology: You can invest in technologies that support your business's growth.

Maintain equipment: You can regularly maintain equipment like forklifts and conveyors to ensure they operate efficiently.

Use the 5S methodology: The 5S methodology focuses on eliminating waste and improving processes.

The 5S methodology is a five-step process for organizing and maintaining a productive warehouse:

- Sort: Remove unnecessary items and evaluate what's needed

- Set in order: Organize essential items for easy access
- Shine: Clean the workspace regularly
- Standardize: Develop uniform procedures and protocols
- Sustain: Maintain and review standards consistently

Estimate your expenses: You can reduce expenses by analyzing your month-to-month needs.

Train and develop employees: Well-trained employees are more efficient and make fewer errors.

Strategy Components

Preliminary data warehouse rollout plan-

-divide warehouse development into phased, successive rollouts

-each rollout focuses on meeting an agreed set of requirements-it is iterative in nature

-manageable

Preliminary data warehouse architecture-

-Define overall data warehouse architecture for pilot and subsequent warehouse rollouts

- ensure scalability of warehouse
- define initial technical architecture of each rollout.

Short-listed data warehouse environment and tools-

- Create a short-list for the tools and environment
- selection should be according to warehousing needs

Warehouse /management and Support Processes:

- Designed to address aspects of planning and managing a data warehouse project
- These are critical to successful implementation and subsequent extension
- processes are defined to assist project manager and warehouse driver during warehouse development projects.
- Following issues are covered in it:

1. Define Issue Tracking and Resolution Process
2. Perform Capacity Planning
3. Define Warehouse Purging Rules
4. Define Security Measures
5. Define Backup and Recovery Strategy
6. Set Up Collection of Warehouse Usage Statistics

1. Define Issue Tracking and Resolution Process

During course of a project a number of business and technical issues will surface. A sample **issue log** tracks all issues that arise during project. Issue logs formalize issue resolution process. They serve as a formal record of key decisions made throughout the project.

No.	Issue Description	Urgency	Raised By	Assigned To	Date Opened	Date Closed	Resolved By	Resolution Description
1	Conflict over definition of "Customer"	High	MWH	MCD	Feb 03	Feb 05	CEO	Use CorPlan's definition
2	Currency exchange rates are not tracked in GL	High	MCD	RGT	Feb 04	-	-	-

Figure 7–1 Sample Issue Log

Some issue tracking guidelines are:-

- **Issue description:** State the issue briefly in two to three sentences.
- **Urgency:** Indicate the priority level of the issue: high, medium, or low.
- **Raised by:** Identify the person who raised the issue.
- **Assigned to:** Identify the person on the team who is responsible for resolving the issue.
- **Date opened:** This is the date when the issue was first logged.
- **Date closed:** This is the date when the issue was finally resolved.
- **Resolved by:** The person who resolved the issue.
- **Resolution description:** State briefly the resolution of this issue in two or three sentences.

2. Perform Capacity Planning

i. Space Requirements

Space requirements are determined by the following:

- schema design, expected volume, and expected growth rate;
- indexing strategy used;
- backup and recovery strategy;
- aggregation strategy;
- staging and duplication area required; and
- metadata space requirements.

ii. Machine Processing Power

-MPP (massively parallel processing) and SMP (symmetric multiprocessing) machines are ideal Choose a configuration that is scalable and that meets minimum processing requirements.

iii. Network Bandwidth

The network bandwidth must not be allowed to slow down the warehouse extraction and warehouse performance. Verify all assumptions about the network bandwidth before proceeding with each rollout.

iv. Number of concurrent user

By effectively managing the number of concurrent users and ensuring adequate resources and system optimization, you can ensure that the data warehouse delivers consistent, high performance even under heavy load conditions.

3. Define Warehouse Purging Rules

Purging rules specify when data are to be removed from the data warehouse.

- Companies are interested in tracking performance over 3-5 to years.
- In cases where a longer retention period is required, the end users will require only high-level summaries for comparison.
- Define the mechanisms for archiving or removing older data from the data warehouse.
- Check for any legal, regulatory, or auditing requirements that may warrant the storage of data in other media prior to actual purging from the warehouse.
- Acquire the software and devices that are required for archiving.

4. Define Security Measures

Keep the data warehouse secure to prevent the loss of competitive information either to unforeseen disasters or to unauthorized users.

Define the security measures for the data warehouse, taking into consideration both physical security (i.e., where the data warehouse is physically located), as well as user- access security.

5. Define Backup and Recovery Strategy

- Consider the following factors:
- Data to be backed up.
- Batch window of the warehouse.
- Maximum acceptable time for recovery.
- Acceptable costs for backup and recovery.
- Also consider the following when selecting the backup mechanism:
- Archive format.
- Automatic backup devices.
- Parallel data streams.
- Incremental backups.
- Offsite backups.
- Backup and recovery procedures.

6. Set Up Collection of Warehouse Usage Statistics

- These are collected to provide data warehouse designer with inputs for further refining the data warehouse design
- to track general usage and acceptance of warehouse.
- Define mechanism for collecting these statistics
- assign resources to monitor and review these regularly.

Support processes in warehouse management are essential for the proper functioning of core business processes. Some examples of support processes include:

IT systems

If IT systems aren't working properly, core business processes can't access the information they need to operate.

HR processes

If HR processes aren't effective, the organization can't attract and retain the best talent.

Financial processes

If financial processes aren't sound, the organization can't manage its finances effectively.

Warehouse management systems (WMS)

WMS systems can support the use of RFID technology and integration with billing and other software.

This can help with automatic receiving, validation, and reconciliation.

Some other warehouse management processes include:

Receiving: Inspecting incoming shipments, checking for damages, and updating inventory records

Put-away: Assigning specific bins or locations within the warehouse for proper storage

Picking: Retrieving items from their designated locations for shipment

Packing: Preparing items for shipment by packing them safely and securely

Shipping: Preparing packages for delivery, generating shipping labels, and coordinating shipments

Warehouse Planning and Implementation

- It is conducted to define scope of one data warehouse rollout.
- combination of top-down and bottom-up tracks gives planning process best of both worlds-a requirements-driven approach that is grounded on available data.
- clear separation of front-end and back-end tracks encourages development of warehouse subsystems for extracting, transporting, cleaning, and loading independently of front-end tools.
- four tracks converge when a prototype of warehouse is created and when actual warehouse implementation takes place.
- Each rollout repeatedly executes four tracks :-

  - top-down
  - bottom-up
  - back-end
  - front-end

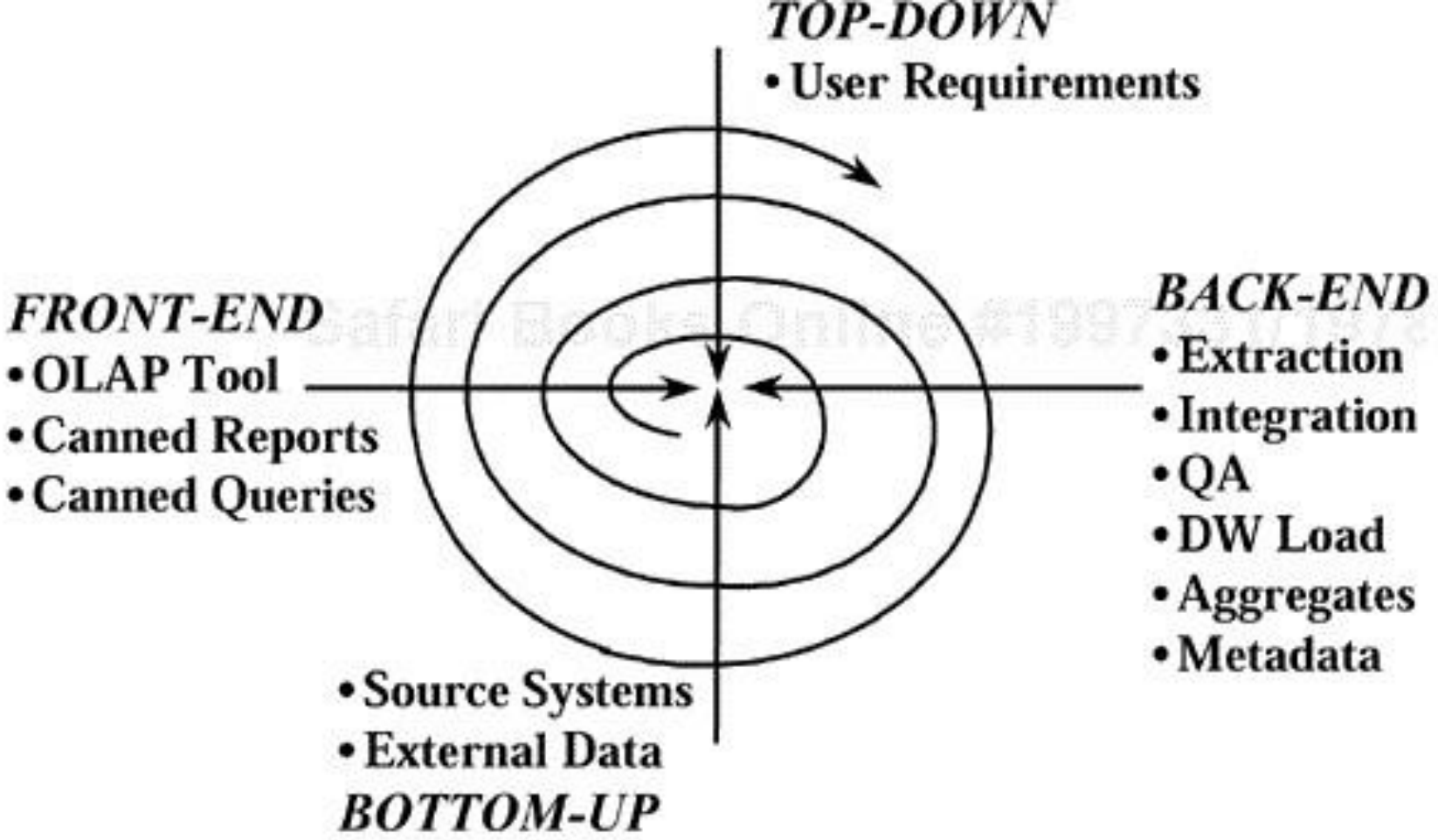


Figure 8-3. The Four Tracks in Warehouse Development

Activities in Data Warehouse Planning

1. Assemble and Orient Team:

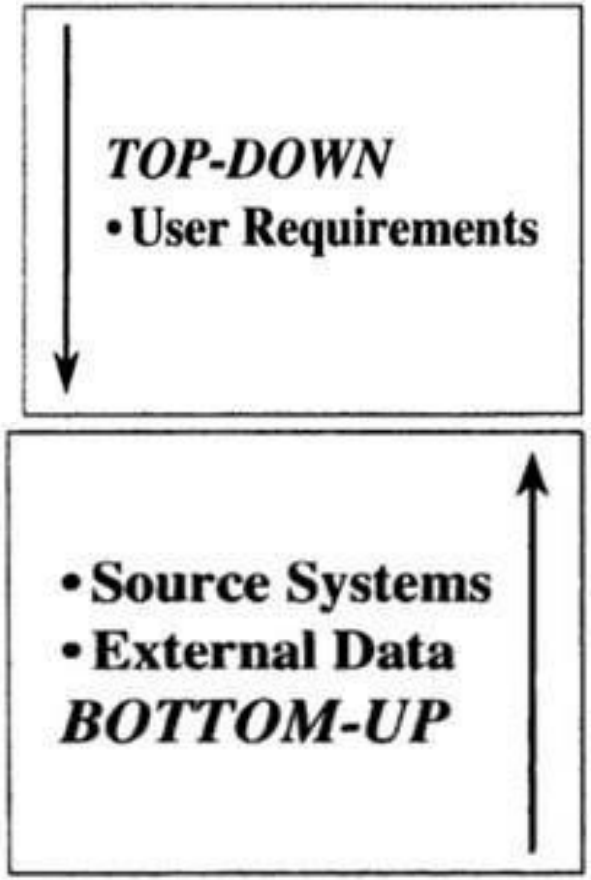
- Identify all parties who will be involved in DW implementation
- brief them about the project.

2. Conduct Decisional Requirements Analysis:

- Analyze to gain understanding of information needs of decision-makers.
- top-down aspect of data warehousing.

3. Conduct Decisional Source System Audit:

- It is a survey of all information systems that are potential sources of data
- Data sources are primarily internal.
- If external data sources are available, they may be integrated into warehouse.



4. Design Logical and Physical Warehouse Schema:

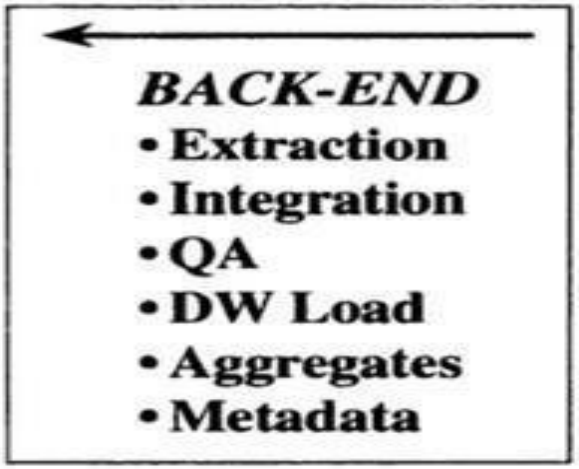
Design data warehouse schema that best meet information requirements of this rollout.

Two schema design techniques are:

Normalization: database schema is designed using the normalization techniques traditionally used for OLTP applications; **Dimensional modeling:** produces de-normalized, star schema designs consisting of fact and dimension tables.

-snowflake schema

-star schema



5. Produce Source-to-Target Field Mapping:

The Source-To-Target Field Mapping documents how fields in the operational (source) systems are transformed into data warehouse fields.

- To eliminate any confusion as to how data are transformed
- data items are moved from source systems to warehouse database
- create a source-to-target field mapping that maps each source field in each source system to target field in DW
- This is required for each field in the source-to-target field mapping.
- critical to successful development and maintenance of DW
- mapping serves as basis for extraction and transformation

		TARGET	No.	1	2	3	4	5	6	7
			Schema	R 1	R 1	R 1	R 1	R 1	R 1	R 1
SOURCE			Table	TT1	TT1	TT1	TT2	TT2	TT2	TT2
No.	System	Table	Fields	TF1	TF2	TF3	TF4	TF5	TF6	TF7
1	SS1	ST1	SF1							
2	SS1	ST1	SF2							
3	SS1	ST1	SF3							
4	SS1	ST1	SF4							
5	SS1	ST2	SF5							
6	SS1	ST2	SF6							
7	SS2	ST2	SF7							
8	SS2	ST3	SF8							
9	SS2	ST3	SF9							
10	SS2	ST3	SF10							
...	...	...	...	...	...	...	...	...	...	...

SOURCE: SS1 = Source System 1, ST 1 = Source Table 1, SF1 = Source Field 1

TARGET: R1 = Rollout 1, TT1 = Target Table 1, TF1 = Target Field 1

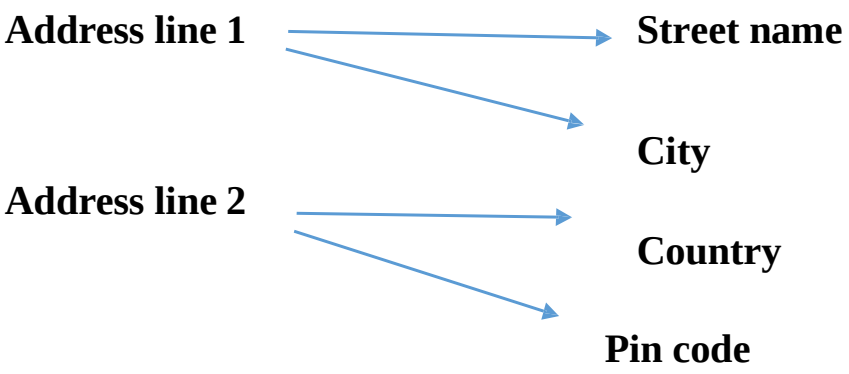
Figure 8-1. Sample Source-to-Target Field Mapping

Example :Mapping

Mapping indicator	Change description	Key indicator	Source Table/File Name	Source Field name	Source Field Length	Source Data Type	Source Field Description	Business Rule	Target Table Name	Target Field Name	Target Data Type	Target Field Length
A		NA	Employee_info	Emp First Name	50	Text	Employee First Name	Direct Mapping	EMP_DIM	EMP_F_NME	Varchar	255
A		NA	Employee_info	Emp Last Name	50	Text	Employee Last Name	Direct Mapping	EMP_DIM	EMP_L_NME	Varchar	255
C	Business Rule Change	PK	Employee_info	EMP No	10	Number	Employee Number	Append "E" before the number	EMP_DIM	EMP_NUM	Varchar	15
A		NA	Employee_info	EMP Dep	50	Text	Employee Department	If Emp Dep="Sales" then load "S" Emp Dep="Packing" then load "P" Emp Dep="Transport" then load "T"	EMP_DIM	EMP_D_IND	Varchar	1

Many-to-many Mapping

- A single field in the data warehouse may be populated by data from more than one source system. This is due to integration of data from multiple sources.
- A field called Customer Name or Product Name will be populated by data from more than one system.
- A single field in OS may need to be split into several fields
- Other examples are numeric figures or balances that have to be allocated correctly to two or more different fields.



Historical Data and Evolving Data Structures

If users require loading of historical data two things are determined :

Changes in schema:

- Determine if schemas of all source systems have changed over the relevant time period.

- For example, if the retention period of the data warehouse is 2 years and data from past 2 years have to be loaded , team must check for possible changes in source system schemas over past two years.
- If schemas have changed over time, the task of extracting the data immediately becomes more complicated.
- Each different schema require a different source-to-target field mapping.

Availability of historical data:

- Determine also if historical data are available for loading.
- Backups during the relevant time period may not contain the required data items.
- Verify assumptions about the availability and suitability of backups for historical data loads.

6. Select Development and Production Environment and Tools:

Finalize the computing environment and tool set for this rollout. If an exhaustive study and selection had been performed during the strategy definition stage, this activity becomes optional.

7. Create Prototype for This Rollout:

Using the short-listed or final tools and production environment, create a prototype of the data warehouse.

8. Create Implementation Plan of This Rollout:

-With scope now fully defined and the source-to-target field mapping fully specified

- it is now possible to draft an implementation plan for this rollout.

9. Warehouse Planning Tips and Caveats:

- The actual data warehouse planning activity will rarely be astraightforward exercise.
- Before conducting your planning activity, understand the concept of Data Trail, Limitations Imposed by Currently Available Data.

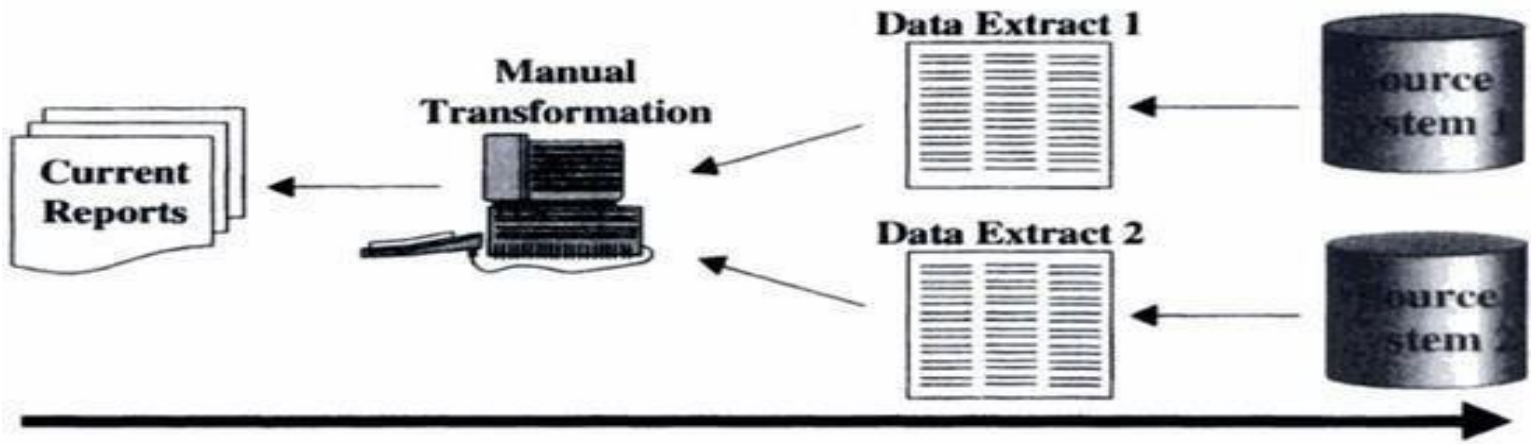


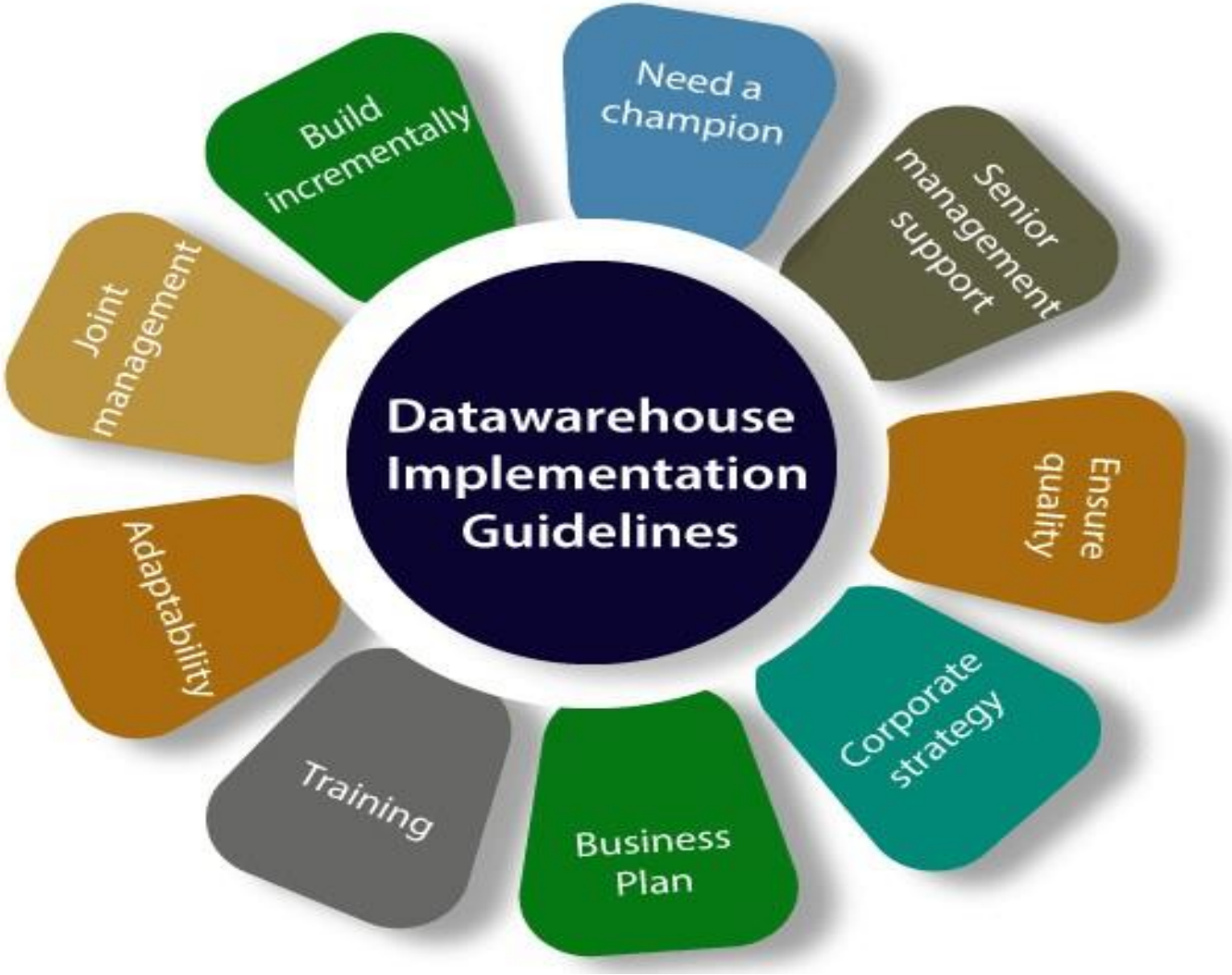
Figure 8–2 Follow the Data Trail

Warehouse Implementation

1. **Requirements analysis and capacity planning:** The first process in data warehousing involves defining enterprise needs, defining architectures, carrying out capacity planning, and selecting the hardware and software tools. This step will contain be consulting senior management as well as the different stakeholder.

2. **Hardware integration:** Once the hardware and software has been selected, they require to be put by integrating the servers, the storage methods, and the user software tools.
3. **Modeling:** Modelling is a significant stage that involves designing the warehouse schema and views. This may contain using a modeling tool if the data warehouses are sophisticated.
4. **Physical modeling:** For the data warehouses to perform efficiently, physical modeling is needed. This contains designing the physical data warehouse organization, data placement, data partitioning, deciding on access techniques, and indexing.
5. **Sources:** The information for the data warehouse is likely to come from several data sources. This step contains identifying and connecting the sources using the gateway, ODBC drives, or another wrapper.
6. **ETL:** The data from the source system will require to go through an ETL phase. The process of designing and implementing the ETL phase may contain defining a suitable ETL tool vendors and purchasing and implementing the tools. This may contains customize the tool to suit the need of the enterprises.
7. **Populate the data warehouses:** Once the ETL tools have been agreed upon, testing the tools will be needed, perhaps using a staging area. Once everything is working adequately, the ETL tools may be used in populating the warehouses given the schema and view definition.
8. **User applications:** For the data warehouses to be helpful, there must be end-user applications. This step contains designing and implementing applications required by the end-users.
9. **Roll-out the warehouses and applications:** Once the data warehouse has been populated and the end-client applications tested, the warehouse system and the operations may be rolled out for the user's community to use.

Warehouse Implementation Guidelines



1. **Build incrementally:** Data warehouses must be built incrementally. Generally, it is recommended that a data marts may be created with one particular project in mind, and once it is implemented, several other sections of the enterprise may also want to implement similar systems. An enterprise data warehouses can then be implemented in an iterative manner allowing all data marts to extract information from the data warehouse.

2. **Need a champion:** A data warehouses project must have a champion who is active to carry out considerable researches into expected price and benefit of the project. Data warehousing projects requires inputs from many units in an enterprise and therefore needs to be driven by someone who is needed for interacting with people in the enterprises and can actively persuade colleagues.
3. **Senior management support:** A data warehouses project must be fully supported by senior management. Given the resource-intensive feature of such project and the time they can take to implement, a warehouse project signal for a sustained commitment from senior management.
4. **Ensure quality:** The only record that has been cleaned and is of a quality that is implicit by the organizations should be loaded in the data warehouses.
5. **Corporate strategy:** A data warehouse project must be suitable for corporate strategies and business goals. The purpose of the project must be defined before the beginning of the projects.
6. **Business plan:** The financial costs (hardware, software, and peopleware), expected advantage, and a project plan for a data warehouses project must be clearly outlined and understood by all stakeholders. Without such understanding, rumors about expenditure and benefits can become the only sources of data, subversion the projects.
7. **Training:** Data warehouses projects must not overlook data warehouses training requirements. For a data warehouses project to be successful, the customers must be trained to use the warehouses and to understand its capabilities.
8. **Adaptability:** The project should build in flexibility so that changes may be made to the data warehouses if and when required. Like any system, a data warehouse will require to change, as the needs of an enterprise change.
9. **Joint management:** The project must be handled by both IT and business professionals in the enterprise. To ensure that proper communication with the stakeholder and which the project is the target for assisting the enterprise's business, the business professional must be involved in the project along with technical professionals.

Hardware and Operating Systems for Data Warehousing

Hardware and operating systems make up the computing environment for your data warehouse. All the data extraction, transformation, integration, and staging jobs run on the selected hardware under the chosen operating system. When you transport the consolidated and integrated data from the staging area to your data warehouse repository, you make use of the server hardware and the operating system software. When the queries are initiated from the client workstations, the server hardware, in conjunction with the database software, executes the queries and produces the results. Choosing the right hardware and operating systems for data warehousing is critical to ensuring optimal performance, scalability, and reliability. Here's a breakdown of considerations for both hardware and operating systems:

Hardware Considerations:

Processing Power:

- Multi-core processors: Data warehouses benefit from parallel processing capabilities offered by multi-core CPUs.
- High clock speeds: Faster processors can handle complex queries and data transformations more efficiently.

Memory (RAM): Sufficient RAM allows for faster data access and query processing.

- In-memory processing: Consider systems with large amounts of RAM or in-memory databases for improved performance.

Storage:

- High-performance storage: Use solid-state drives (SSDs) or high-speed storage arrays for fast data access.
- Scalable storage: Ensure the storage system can scale with growing data volumes.

Network:

High-speed network connections: Fast network infrastructure minimizes data transfer latency between components of the data warehouse architecture.

Redundancy: Implement redundant network connections to ensure high availability and fault tolerance.

Scalability:

Scalable architecture: Choose hardware that supports horizontal scaling to accommodate growing data and user loads.

Distributed processing: Consider distributed computing frameworks like Apache Hadoop or Spark for scalable processing.

Data Redundancy and Fault Tolerance:

RAID configurations: Use RAID (Redundant Array of Independent Disks) for data redundancy and fault tolerance.

Backup systems: Implement regular backups and disaster recovery solutions to protect against data loss.

Hardware Acceleration:

GPU acceleration: Graphics processing units (GPUs) can accelerate certain data processing tasks, such as machine learning algorithms and complex analytics.

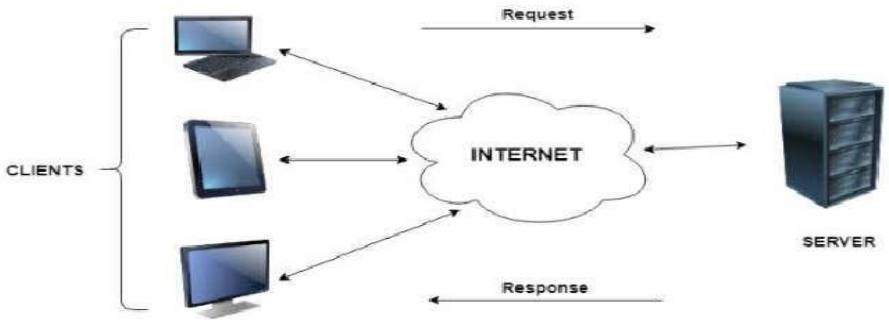
Operating System Considerations

- **Compatibility**:
  - Ensure compatibility with the chosen database management system (DBMS) and other software components of the data warehouse stack.
- **Performance**:
  - Choose operating systems known for stability, performance, and reliability.
  - Linux distributions like CentOS, Red Hat Enterprise Linux (RHEL), or Ubuntu Server are popular choices for data warehousing due to their stability and performance.
  - Implement access controls, firewalls, and encryption to secure data and infrastructure.
- **Security**:

  - Select an operating system with robust security features and regular updates to protect against vulnerabilities.
- **Manageability**:
  - Choose an operating system with robust management tools and support for automation.
  - Consider systems with centralized management capabilities for easier administration of multiple servers.
- **Compatibility with Tools and Software**:
  - Ensure compatibility with data warehousing software, ETL tools, monitoring tools, and other components of the data warehouse ecosystem.
- **Scalability and Resource Management**:
  - Operating systems should support resource management features like process scheduling, memory management, and disk I/O optimization to ensure efficient resource utilization.
- **Virtualization and Containerization**:
  - Consider virtualization or containerization technologies like VMware, Docker, or Kubernetes for flexible deployment and resource allocation.

Choosing the right hardware and operating systems is essential for building a high-performance, scalable, and reliable data warehouse infrastructure. Considerations include processing power, memory, storage, network, scalability, compatibility, security, manageability, and support for virtualization/containerization. By carefully evaluating these factors and aligning them with the organization's requirements, you can build a robust data warehousing environment that meets current and future needs.

Client/Server Computing Model & Data Warehousing



The client/server computing model plays a significant role in the architecture of data warehousing systems, providing a framework for distributing processing tasks between clients (end-users) and servers (data warehouse infrastructure). Here's how the client/server model relates to data warehousing:

In the client/server computing model, computing tasks are divided between clients and servers:

- **Client**:
  - Refers to the end-user devices, such as desktop computers, laptops, tablets, or smartphones.
  - Executes user applications and interfaces with the user.
  - Requests data and services from servers.
- **Server**:
  - Refers to powerful computers or server clusters that provide services to clients.
  - Handles data storage, processing, and management tasks.
  - Responds to client requests by providing data or executing operations.
- Role in Data Warehousing
- In the context of data warehousing, the client/server model is used to facilitate access to and utilization of the data warehouse by end-users:

- **Client-Side Applications**:
  - Business intelligence (BI) tools, reporting applications, and analytical software installed on client devices.
  - Allows users to interact with the data warehouse, query data, generate reports, and visualize insights.
- **Server-Side Components**:
  - Data warehouse servers and associated infrastructure.
  - Stores and manages large volumes of structured and sometimes unstructured data.
  - Executes complex data processing tasks, including ETL (Extract, Transform, Load), data aggregation, and query optimization.

The client/server model facilitates the interaction between end-users and the data warehouse:

- **Data Retrieval**:
  - Clients send queries to the data warehouse servers to retrieve specific datasets or perform analyses.
  - Servers process these queries, accessing the required data from storage, and returning the results to the clients.
- **Data Presentation**:
  - Servers provide processed data to clients in a format suitable for analysis or visualization.
  - Clients use BI tools or reporting applications to present the data to end-users in the form of charts, graphs, tables, or dashboards.
- **Data Manipulation**:
  - Clients can manipulate and analyze data locally using client-side applications.
  - Servers may also provide services for advanced data processing, such as machine learning or predictive analytics, depending on the architecture.

Benefits

- The client/server computing model offers several benefits in the context of data warehousing:
- **Scalability**:
  - Enables scaling of both client and server components independently based on demand.

- Additional clients can be added without impacting server performance, and vice versa.

- **Centralized Management**:
  - Centralizes data storage and management on servers, simplifying administration and ensuring data consistency.
- **Resource Utilization**:
  - Distributes processing tasks between clients and servers, optimizing resource utilization and performance.
- **Flexibility**:
  - Allows users to access data warehouse resources from various client devices and locations, providing flexibility in data access and analysis.

The client/server computing model serves as the architectural foundation for data warehousing systems, enabling efficient interaction between end-users and data warehouse servers. By dividing computing tasks between clients and servers, this model supports scalable, centralized, and flexible data access and analysis, essential for modern data-driven organizations.

Parallel Processors & Cluster Systems

Parallel processing and cluster systems can be used in warehousing to improve processing time and scalability, and to handle complex queries and large amounts of data:

Parallel processing

- DW is a user-centric and query-intensive environment where users will constantly be executing complex queries
- Each query need large volumes of data to produce result sets.
- If the data warehouse is not tuned properly for handling large, complex, simultaneous queries efficiently, the value of the data warehouse will be lost. Performance is of primary importance.

- To speed up query processing, data loading, and index creation this is used
- Split problem in smaller tasks that are executed concurrently.

Advantages:

Increasing speed & optimizing resources utilization

Disadvantages:

Complex programming models – difficult development

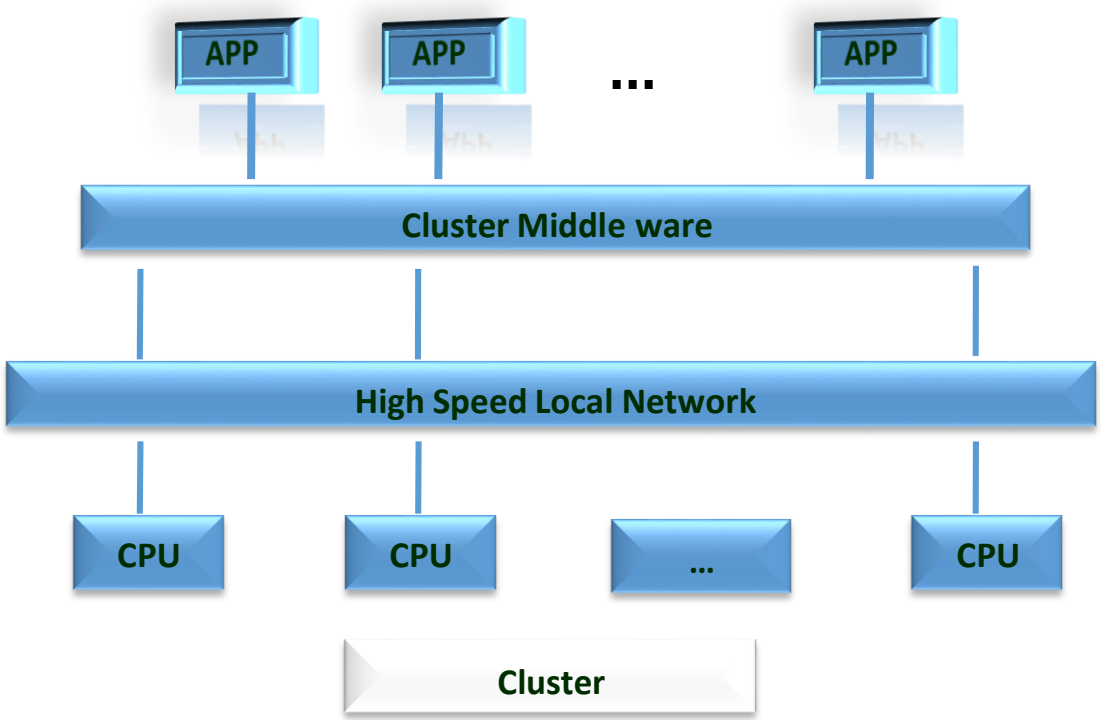
Cluster Systems

A computer cluster is a group of linked computers together.

- components of a cluster are connected through fast local area networks.
- deployed to improve performance and availability
- In such environments, each PU executes a copy of a standard operations and inter-PU communications are performed over an open system based interconnect(Ethernet or TCP/IP)

<u>Cluster consists of:</u>

- Nodes(master+computing)
- Network
- OS
- Cluster middleware: Middleware such as MPI which permits compute clustering programs to be portable to a wide variety of clusters


APP
APP
...
APP
Cluster Middle ware
High Speed Local Network
CPU
CPU
...
CPU
Cluster

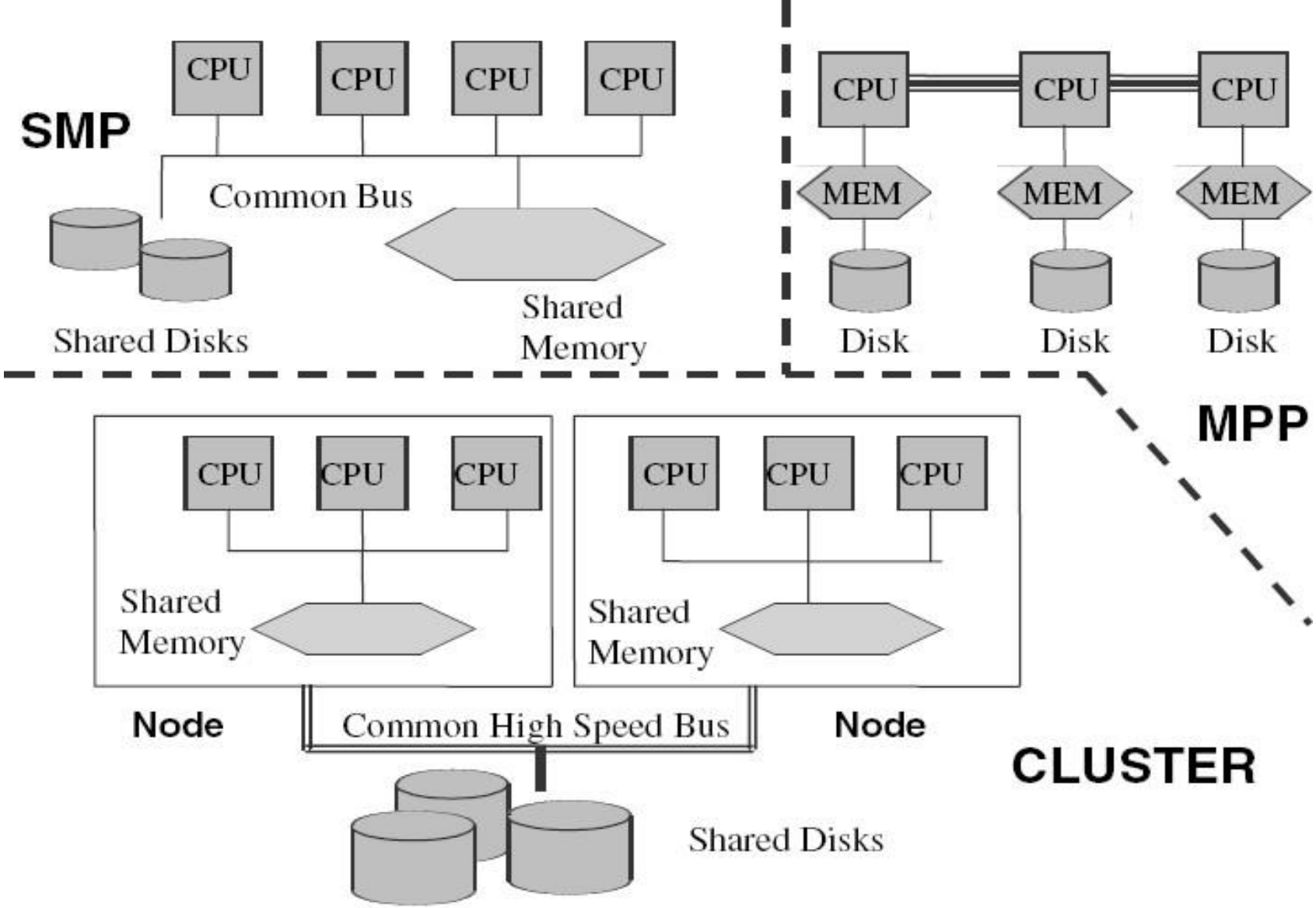


Figure 3-6 Parallel processing: hardware options.

Some hardware examples are:-

- Digital-64-bit AlphaServers and Digital Unix or Open VMS. Both SMP and MPP

- HP-HP 9000 Enterprise Parallel Server.
- IBM-RS6000 ,AIX OS have been positioned for data warehousing
- AS/400 -used for data mart implementations
- Microsoft- -Windows NT operating system us successful for datamart deployments.
- Sequent-Sequent NUMA-Q and the DYNIX operating system.

Parallel processing software perform following steps:

1. Analyzing a large task to identify independent units that can beexecuted in parallel
2. Identifying smaller units that must be executed one after the other
3. Executing independent units in parallel and the dependent units in the proper sequence
4. Collecting, collating, and consolidating results returned by smaller units

- Parallel server option allows each node to have its own separate database instance, and enables all database instances to access a common set of underlying database files.
- parallel query option supports key operations such as query processing, dataloading, and index creation to be parallelized.

Here are some other benefits of parallel processing and cluster systems:

Improved scalability

Parallel processing and cluster systems can scale to handle larger and more complex problems.

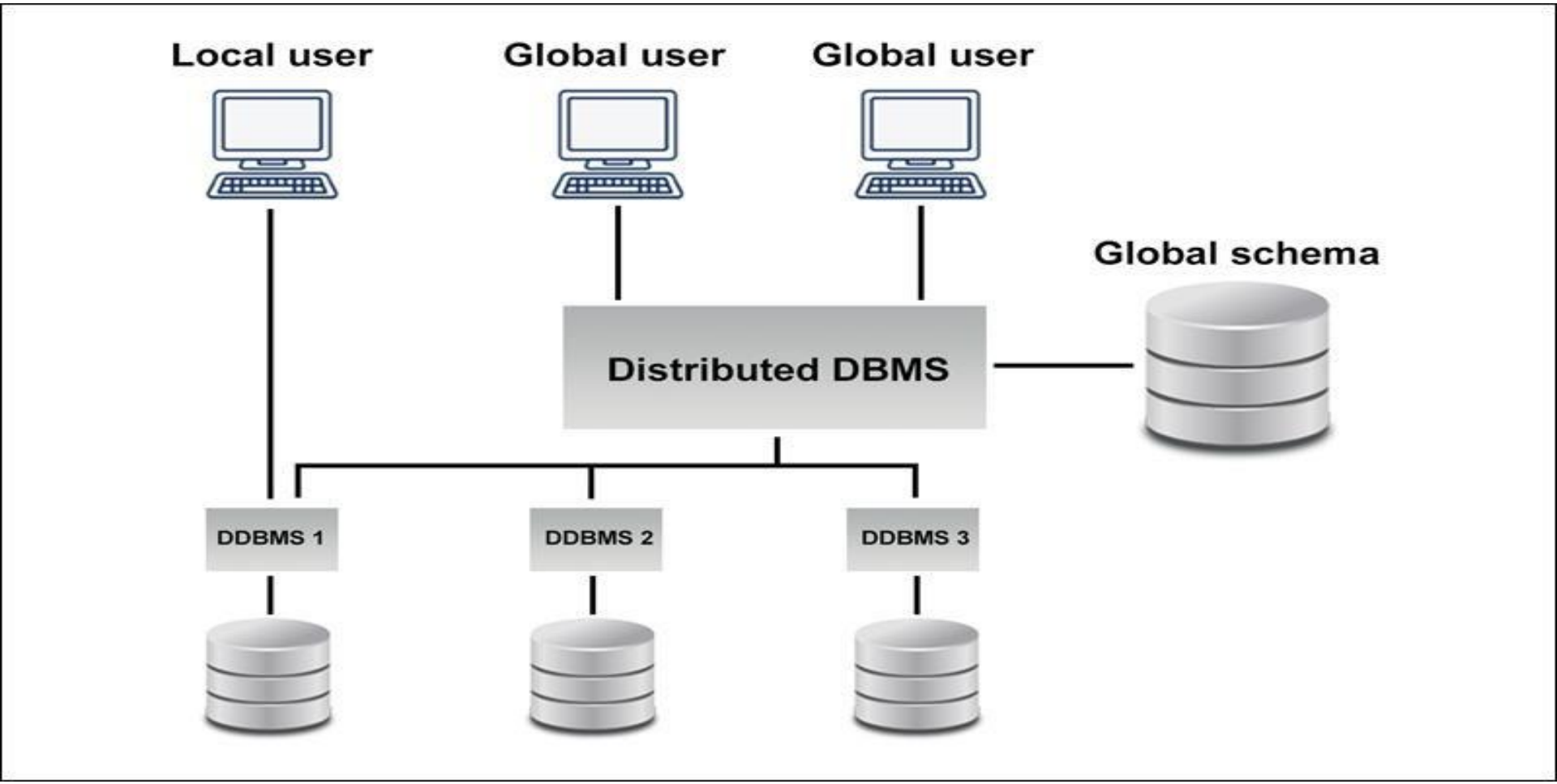
Better performance

Parallel processing can improve performance by segmenting a problem and processing each part in parallel.

Communication and coordination

Parallel computing requires efficient communication and coordination between nodes, which can be managed by a parallel programming framework.

Distributed DBMS implementations



A distributed database system is a type of database management system that stores data across multiple computers or sites that are connected by a network.

A distributed database management system (DDBMS) manages a distributed database (DDB) by making it appear as a single logical database to users. A DDB is a collection of databases that are spread across multiple sites in a network, and a DDBMS manages it as if the data were stored in a single location.

Here are some ways a DDBMS works:

Data distribution: A DDB's data is split and replicated across multiple sites. This process is sometimes called data partitioning.

Data synchronization: A DDBMS periodically synchronizes data across all sites.

Data updates: Updates made to data in one location are automatically reflected in other locations.

Data integrity: A DDBMS maintains the confidentiality and integrity of the data.

Data distribution can be done in two ways:

Horizontal partitioning: Splits data tables into rows across multiple nodes.

Vertical partitioning: Splits tables into columns across multiple nodes.

Horizontal and vertical partitioning are two ways to split data in a database to create multiple smaller databases, also known as sharding:

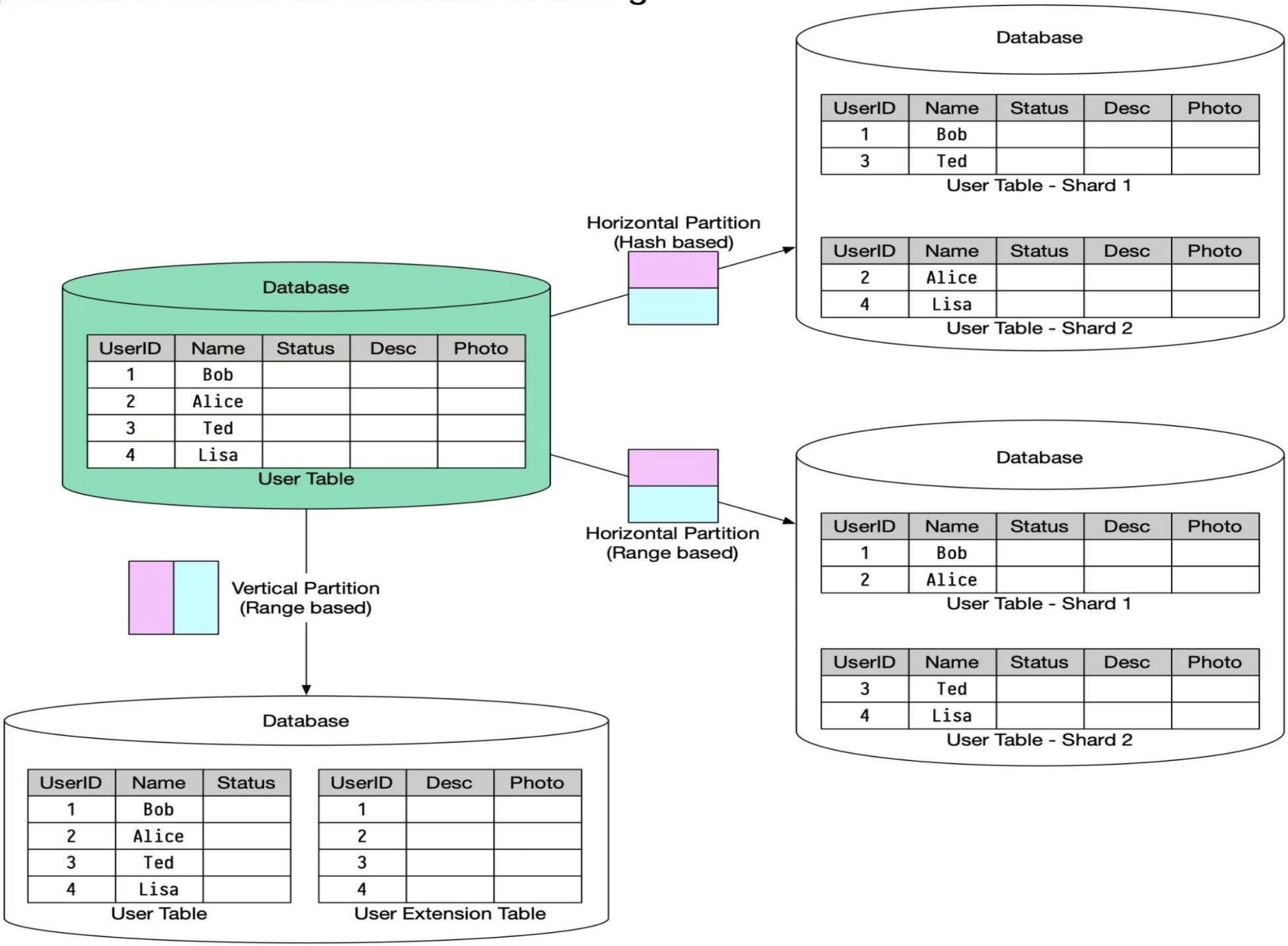
Horizontal partitioning

Also known as sharding, this strategy splits data by rows, with each partition containing a subset of rows. All partitions have the same schema, or columns. This method makes it easier to manage large datasets.

Vertical partitioning

This strategy splits data by columns, with each partition containing a subset of columns. The columns are divided based on how often they are accessed. This method optimizes access to columns that are frequently queried together.

Vertical & Horizontal Database Sharding



Warehousing Software

Warehouse management software (WMS) is a tool that helps companies manage and control their warehouse operations. It helps with inventory management, order fulfillment, and other warehouse processes. WMS can help companies improve their warehouse efficiency and productivity. Here are some things that WMS can do:

Inventory management

WMS can track inventory levels in real time, including items in transit and in stores. It can also help with cycle counting and demand forecasting.

Order fulfillment

WMS can help streamline order fulfillment. It can also help with picking and packing goods.

Labor management

WMS can help warehouse managers monitor worker performance. It can also help with task interleaving to minimize workers' travel time.

Shipping

WMS can help with generating packing lists and invoices, and sending advance shipment notifications.

Yard and dock management

WMS can help truck drivers find the right loading docks.

Reporting

WMS can help managers analyze warehouse performance and find areas for improvement.

Warehouse Schema Design

Designing a data warehouse schema involves understanding business requirements and analytical needs to choose between Star, Snowflake, or Galaxy schema types. Key steps include identifying critical business processes, defining fact tables for measurable metrics, and dimension tables for descriptive context.

When designing a data warehouse schema, you can consider the following steps:

Understand business needs: Consider the business goals and analytical needs of different departments.

Identify key processes: Determine the key business processes and measures to be analyzed.

Choose a schema type: Select a schema type, such as star, snowflake, or galaxy, that best fits the data warehouse's objectives and characteristics.

Define tables: Create fact tables for measurable metrics and dimension tables for descriptive context.

Ensure data integrity: Use primary and foreign keys to maintain relationships.

Optimize for performance: Use indexes, partitions, aggregations, and denormalizations to optimize the schema design for performance.

Document and validate: Document and validate the schema design with stakeholders and users.

Plan for scalability: Ensure scalability for future growth.

Plan for ETL processes: Plan for efficient Extract, Transform, and Load (ETL) processes.